🎓 **Aula 7 –** Início da API Real em TypeScript

🧠 Início da API Real em TypeScript

🔥 Mudança de nível

- Antes: funções isoladas
- Agora: aplicação organizada

“Agora complicou 😱, será?”

Para que serve uma API?

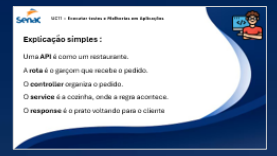
- Recebe requisições
- Processa dados
- Retorna respostas

🧪 Exemplo simples

- 👉 cadastrar usuário
- 👉 listar usuários
- 👉 buscar usuário por id

🧠 Estrutura da aplicação

- Routes
- Controllers
- Services
- app



🧠 Benefício da separação

- código limpo
- manutenção fácil
- teste mais simples

🧠 **“Sistema organizado é mais fácil de testar e de evoluir”**

🧪 Fluxo Padrão

📄 **Cliente → Rota → Controller → Service → Resposta**

- **Cliente** faz a requisição
- **Rota** recebe o caminho
- **Controller** coordena a entrada e saída
- **Service** aplica a regra de negócio
- **Resposta** volta para quem chamou

🧠 Insight

"Se tudo ficar misturado em um arquivo só, vira bagunça. Quando eu separo responsabilidade, eu facilito manutenção e teste."

🧪 Projeto da Aula – 1° API

Nesta aula, vamos construir uma API simples de usuários com:

- cadastro de usuário
- listagem de usuários
- busca de usuário por id

Sem banco de dados por enquanto. Vamos usar memória para deixar o foco na estrutura.

⚙️ 6. Setup do Projeto

🧠 **Comandos iniciais:**

1° Criar uma pasta api-testes

```
npm init -y
npm install express
npm install typescript ts-node-dev @types/node @types/express -D
npx tsc --init
```

Ajuste do package.json

JSON

```json
{
  "name": "api-testes",
  "version": "1.0.0",
  "scripts": {
    "dev": "ts-node-dev --respawn --transpile-only src/server.ts",
    "test": "jest",
    "test:coverage": "jest --coverage"
  }
}
```

Ajuste do tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "module": "commonjs",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  }
}
```

🗂️ 7. Estrutura de Pastas

```
api-testes/
│
├── src/
│   ├── app.ts
│   ├── server.ts
│   ├── routes/
│   │   └── userRoutes.ts
│   ├── controllers/
│   │   └── UserController.ts
│   ├── services/
│   │   └── UserService.ts
│   └── types/
│       └── User.ts
│
├── package.json
└── tsconfig.json
```

🧠 8. Explicação didática da estrutura

app.ts
Configura o Express
server.ts
Sobe o servidor
Routes
Define as rotas
Controllers
Recebe requisição e devolve resposta
Services
Onde fica a regra de negócio
Types
Define tipos do sistema

9. Código Completo da Aula

src/types/User.ts

```
export interface User {
    id: number;
    name: string;
    email: string;
}
```

9. Código Completo da Aula

src/services/UserService.ts

```typescript
import { User } from '../types/User';
export class UserService {
    private users: User[] = [];
    private currentId = 1;

    createUser(name: string, email: string): User {
        if (!name || !email) {
            throw new Error('Nome e email são obrigatórios');
        }
    const emailJaExiste = this.users.some((user) => user.email === email);
    if (emailJaExiste) {
        throw new Error('Email já cadastrado');
    }
    const newUser: User = { id: this.currentId++,
        name,
        email,
    };
    this.users.push(newUser);
    return newUser;
}

listUsers(): User[] {
    return this.users;
}

findUserById(id: number): User | undefined {
    return this.users.find((user) => user.id === id);
 }
}
```

9. Código Completo da Aula

src/controllers/UserController.ts

```
import { Request, Response } from 'express';
import { UserService } from '../services/UserService';
const userService = new UserService();
export class UserController {
    create(request: Request, response: Response): Response {
        try {
            const { name, email } = request.body;
            const user = userService.createUser(name, email);
            return response.status(201).json(user);
        } catch (error) {
            const message = error instanceof Error ? error.message : 'Erro interno';
            return response.status(400).json({ error: message });
        }
    }
    list(request: Request, response: Response): Response {
        const users = userService.listUsers();
        return response.status(200).json(users);
    } findById(request: Request, response: Response): Response {
        const id = Number(request.params.id);
        const user = userService.findUserById(id);
        if (!user) {
            return response.status(404).json({ error: 'Usuário não encontrado' });
        }
        return response.status(200).json(user);
    } }
}
```

9. Código Completo da Aula

src/routes/userRoutes.ts

```
import { Router } from 'express';
import { UserController } from '../controllers/UserController';

const userRoutes = Router();
const userController = new UserController();

userRoutes.post('/users', (request, response) =>
    userController.create(request, response));
userRoutes.get('/users', (request, response) =>
    userController.list(request, response));
userRoutes.get('/users/:id', (request, response) =>
    userController.findById(request, response));

export { userRoutes };
```

9. Código Completo da Aula

src/app.ts

```
import express from 'express';
import { userRoutes } from './routes/userRoutes';

const app = express();

app.use(express.json());
app.use(userRoutes);

export { app };
```

9. Código Completo da Aula

src/server.ts

```
import { app } from './app';

const PORT = 3000;

app.listen(PORT, () => {
    console.log('Servidor rodando em http://localhost:${PORT}');
});"
```

10. Rodando a API

```
npm run dev
```

🧪 11. Como testar manualmente com Insomnia ou Postman

Criar usuário

POST */users*

```
JSON

{
  "name": "Éder",
  "email": "eder@email.com"
}
```

🧪 11. Como testar manualmente com Insomnia ou Postman

Listar usuários

GET `/users`

Buscar usuário por id

GET `/users/1`

Obrigado!

Prof. Éder Silva

Senac